



TECNICO EN CIBERSEGURIDAD

Participante: Luis Mario Reynoso

Técnico: Ciberseguridad

Nombre del Módulo: CRIPTOGRAFÍA Y CIFRADO DE DATOS

Facilitador: Kevin Feliz Henríquez

Centro: Cyberclass

Nombre Actividad: Laboratorio: Simulador de Cifrado César con Python

Descripción: En esta actividad estaremos creando un programa con el lenguaje Python llamado CryptoCesar Pro, que permite cifrar y descifrar mensajes usando el Cifrado César. Además, incluye historial, contador de operaciones, exportación a archivo TXT y una demostración de cómo otra persona puede descifrar un mensaje con clave o mediante fuerza bruta.

¿Qué construiremos?

Desarrollaremos una aplicación de consola que permita practicar el algoritmo clásico de Cifrado César.

- Introducir un mensaje en texto plano.
- Cifrar el mensaje usando una clave numérica.
- Descifrar un mensaje cuando se conoce la clave.
- Mostrar cómo otra persona podría probar todas las claves mediante fuerza bruta.
- Guardar las operaciones realizadas en un historial temporal.
- Contar cuántas operaciones se han realizado.
- Exportar el historial a un archivo TXT.
- Salir del programa mediante un menú interactivo.

Objetivo del proyecto

Comprender la diferencia entre texto plano y texto cifrado, aplicar cifrado por sustitución y crear una aplicación interactiva en Python usando funciones, ciclos y condicionales.

Introducción a la criptografía

La criptografía es la disciplina que permite proteger información mediante técnicas de transformación de datos. Su objetivo es que un mensaje no pueda ser leído fácilmente por personas no autorizadas.

- Texto plano: mensaje original que cualquier persona puede leer. Ejemplo: HOLA.
- Texto cifrado: resultado de aplicar un algoritmo criptográfico. Ejemplo: KROD.
- Algoritmo: conjunto de pasos ordenados para resolver un problema.
- Clave: valor utilizado para cifrar o descifrar la información.

En el Cifrado César, cada letra se desplaza una cantidad fija de posiciones dentro del alfabeto. Por ejemplo, con una clave de 3, la letra A se convierte en D, B se convierte en E y C se convierte en F.

Original	Cifrado con clave 3
A	D
B	E
C	F
HOLA	KROD

FASE 3: Comprendiendo el Programa

Variable global: **historial**

```
historial = []
```

La variable `historial` es una lista que guarda en memoria las operaciones realizadas por el usuario mientras el programa está abierto.

Cada registro contiene el tipo de operación, el mensaje original, la clave usada y el resultado obtenido.

```
{
    "operacion": "Cifrado",
    "mensaje_original": "HOLA",
    "clave": 3,
    "resultado": "KROD"
}
```

Variable global: **contador_operaciones**

```
contador_operaciones = 0
```

Esta variable se utiliza para contar la cantidad de operaciones realizadas. Cada vez que el usuario cifra, descifra o realiza una fuerza bruta, el contador aumenta en 1.

Función **mostrar_menu()**

```
def mostrar_menu():
    print("\n===== CRYPTOCESAR PRO =====")
    print("1. Cifrar mensaje")
    print("2. Descifrar mensaje")
    print("3. Ataque por fuerza bruta")
    print("4. Ver historial")
    print("5. Ver contador de operaciones")
    print("6. Exportar historial a TXT")
    print("7. Salir")
```

Esta función muestra las opciones principales de la aplicación. El usuario puede elegir qué operación desea realizar.

Función `solicitar_clave()`

```
def solicitar_clave():
    while True:
        clave = input("Digite la clave numerica: ").strip()

        if clave.isdigit():
            return int(clave)
        else:
            print("Error: debe digitar una clave numerica.")
```

Esta función solicita una clave numérica al usuario. Usa un ciclo while True para repetir la pregunta hasta que se introduzca un dato válido.

El método strip() elimina espacios al inicio y al final, mientras que isdigit() verifica si el valor introducido es numérico.

Función `cifrar_cesar(texto, clave)`

Esta función recibe un texto y una clave. Luego desplaza cada letra hacia adelante dentro del alfabeto según el valor de la clave.

```
def cifrar_cesar(texto, clave):
    resultado = ""

    for caracter in texto:
        if caracter.isalpha():
            if caracter.isupper():
                codigo_base = 65
            else:
                codigo_base = 97

            posicion = ord(caracter) - codigo_base
            nueva_posicion = (posicion + clave) % 26
            resultado += chr(nueva_posicion + codigo_base)
        else:
            resultado += caracter

    return resultado
```

El ciclo for recorre cada carácter del texto. Si el carácter es una letra, se cifra. Si no es una letra, se conserva igual.

La función `ord()` convierte una letra en su código numérico ASCII. La función `chr()` convierte un número nuevamente en una letra.

La operación módulo `% 26` permite volver al inicio del alfabeto cuando el desplazamiento pasa de la Z.

Función `descifrar_cesar(texto, clave)`

Esta función permite que otra persona descifre el mensaje si conoce la clave utilizada. En vez de sumar la clave, la resta.

```
def descifrar_cesar(texto, clave):
    resultado = ""

    for caracter in texto:
        if caracter.isalpha():
            if caracter.isupper():
                codigo_base = 65
            else:
                codigo_base = 97

            posicion = ord(caracter) - codigo_base
            nueva_posicion = (posicion - clave) % 26
            resultado += chr(nueva_posicion + codigo_base)
        else:
            resultado += caracter

    return resultado
```

Ejemplo: si el mensaje cifrado es KROD y la clave es 3, al restar 3 posiciones a cada letra se obtiene HOLA.

Función `fuerza_bruta_cesar(texto_cifrado)`

Esta función muestra cómo una persona podría intentar descifrar un mensaje sin conocer la clave. Como el Cifrado César solo tiene 25 claves posibles, el programa prueba todas las claves desde 1 hasta 25.

```
def fuerza_bruta_cesar(texto_cifrado):
    print("\n==== POSIBLES DESCIFRADOS =====")

    for clave in range(1, 26):
        resultado = descifrar_cesar(texto_cifrado, clave)
        print(f"Clave {clave}: {resultado}")
```

Este método se conoce como ataque por fuerza bruta. El atacante revisa los resultados y selecciona el que tenga sentido.

Función `guardar_historial()`

```
def guardar_historial(operacion, mensaje_original, clave,
    resultado):
    historial.append({
        "operacion": operacion,
        "mensaje_original": mensaje_original,
        "clave": clave,
        "resultado": resultado
    })
```

Esta función agrega un nuevo registro a la lista historial. El método `append()` permite añadir información al final de una lista.

Función `mostrar_historial()`

Muestra todas las operaciones guardadas. Primero verifica si el historial está vacío. Si contiene información, utiliza un ciclo for para recorrer cada registro.

```
def mostrar_historial():
    if len(historial) == 0:
        print("\nNo hay operaciones guardadas en el historial.")
    else:
        print("\n==== HISTORIAL DE OPERACIONES =====")

        for indice, registro in enumerate(historial, start=1):
            print(f"\nOperacion #{indice}")
            print(f"Tipo: {registro['operacion']}")
            print(f"Mensaje original: {registro['mensaje_original']}")
            print(f"Clave: {registro['clave']}")
            print(f"Resultado: {registro['resultado']}")
```

La función `enumerate()` permite mostrar un número de orden para cada operación del historial.

Función `exportar_historial()`

Esta función crea un archivo llamado `historial_crypto_cesar.txt` y guarda dentro de él las operaciones realizadas.

```
def exportar_historial():
    if len(historial) == 0:
        print("\nNo hay historial para exportar.")
        return

    archivo = open("historial_crypto_cesar.txt", "w", encoding="utf-8")
    archivo.write("==== HISTORIAL CRYPTOCESAR PRO =====\n")

    for indice, registro in enumerate(historial, start=1):
        archivo.write(f"\nOperacion #{indice}\n")
        archivo.write(f"Tipo: {registro['operacion']}\n")
        archivo.write(f"Mensaje original: {registro['mensaje_original']}\n")
        archivo.write(f"Clave: {registro['clave']}\n")
```

```
        archivo.write(f"Resultado: {registro['resultado']}\n")

archivo.close()
```

El parámetro w indica que el archivo se abre en modo escritura. Si el archivo no existe, Python lo crea automáticamente.

Función principal()

Controla todo el flujo del programa. Utiliza un ciclo while True para mantener la aplicación funcionando hasta que el usuario seleccione la opción de salir.

```
def principal():
    while True:
        mostrar_menu()
        opcion = input("Seleccione una opcion: ").strip()

        if opcion == "1":
            ejecutar_cifrado()
        elif opcion == "2":
            ejecutar_descifrado()
        elif opcion == "3":
            ejecutar_fuerza_bruta()
        elif opcion == "7":
            print("\nGracias por usar CryptoCesar Pro.")
            return
```

La instrucción return finaliza la función principal. Como esta función es la que ejecuta el programa, al retornar se termina la aplicación.

¿Cómo lo descifraría otra persona?

Caso 1: La persona conoce la clave

Si una persona conoce la clave usada para cifrar el mensaje, puede descifrarlo desplazando cada letra hacia atrás la misma cantidad de posiciones.

Texto cifrado	Clave	Texto descifrado
KROD	3	HOLA
XJLZWNIFI	5	SEGURIDAD

Caso 2: La persona no conoce la clave

Si la persona no conoce la clave, puede probar todas las claves posibles. Esto se llama fuerza bruta. En el Cifrado César solo existen 25 claves útiles, por eso es fácil romperlo.

Mensaje cifrado: KROD

Clave 1: JQNC

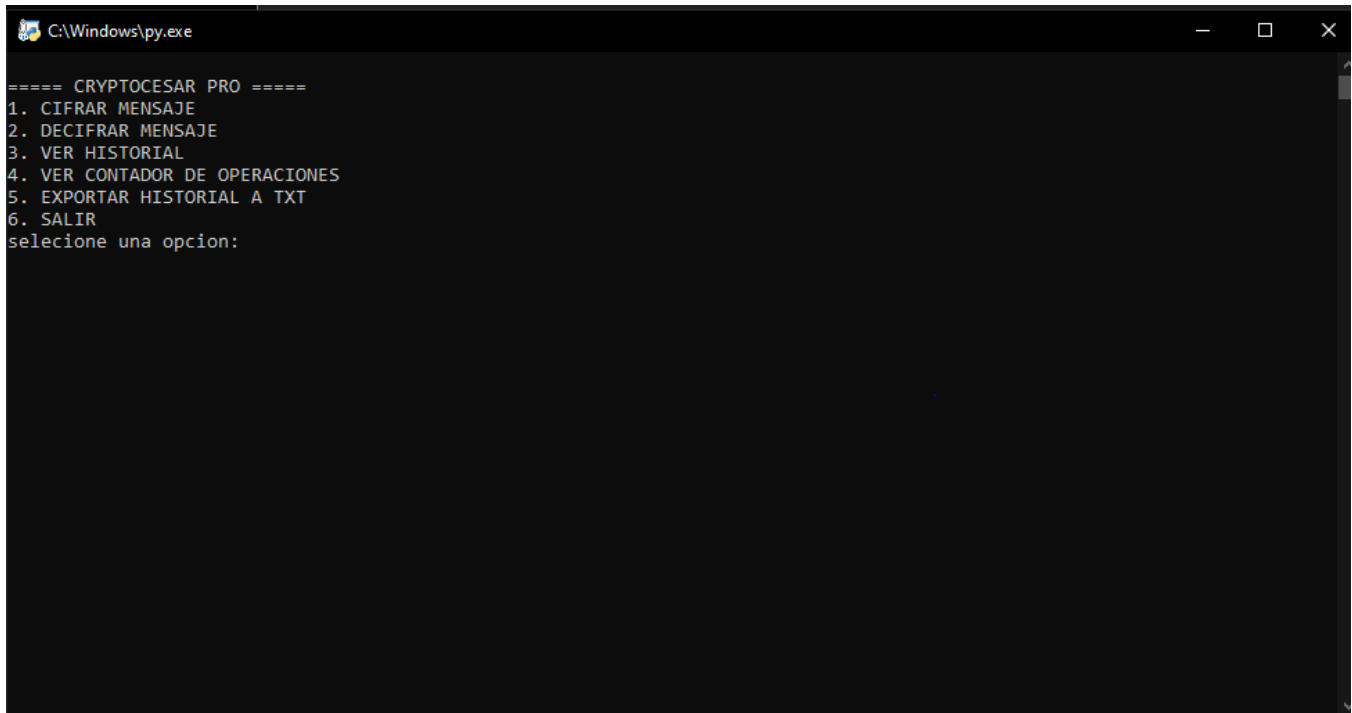
Clave 2: IPMB

Clave 3: HOLA

Clave 4: GNKZ

...

Al ver los resultados, la persona puede identificar que la clave correcta es 3 porque aparece un mensaje legible: HOLA.



```
C:\Windows\py.exe

===== CRYPTOCESAR PRO =====
1. CIFRAR MENSAJE
2. DECIFRAR MENSAJE
3. VER HISTORIAL
4. VER CONTADOR DE OPERACIONES
5. EXPORTAR HISTORIAL A TXT
6. SALIR
seleccione una opcion:
```

Conclusión

El proyecto CryptoCesar Pro permite comprender de forma práctica cómo funciona un algoritmo criptográfico clásico. A través del Cifrado César se aprende la diferencia entre texto plano y texto cifrado, el uso de claves, el proceso de cifrado y el proceso de descifrado.

También se demuestra una debilidad importante: si una persona no conoce la clave, puede probar todas las combinaciones posibles mediante fuerza bruta. Por esta razón, el Cifrado César se utiliza actualmente con fines educativos y no para proteger información real.

El programa cumple con los objetivos de la práctica porque utiliza funciones, ciclos while, ciclos for, condicionales, almacenamiento en memoria, contador de operaciones y exportación de resultados.